

Shrinkage methods and PCR

Shrinkage methods

- ▶ In the subset selection methods we use only a, well. . . , subset of the predictors
- ▶ With shrinkage methods, we use all predictors - but to a different degree
- ▶ OLS: Minimize $RSS(\beta) = \sum_{i=1}^n (y_i - \mathbf{x}_i' \beta)^2$
- ▶ Ridge regression: Minimize $RSS(\beta) + \lambda \sum_{k=1}^p \beta_k^2$
- ▶ LASSO: Minimize $RSS(\beta) + \lambda \sum_{k=1}^p |\beta_k|$
- ▶ Why would we improve predictions by adding the extra terms in Ridge regression and LASSO?

Ridge regression

- ▶ Ridge regression: Minimize

$$\underbrace{\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2}_{\text{Fit to data}} + \underbrace{\lambda \sum_{k=1}^p \beta_k^2}_{\text{Shrinkage penalty}}$$

- ▶ The shrinkage term penalizes values of $\beta_1, \dots, \beta_p$ that are far away from zero.
- ▶ Bias-variance trade-off: A small $RSS(\beta)$ -term means a good fit but might be due to overfitting. The shrinkage penalty controls for that.
- ▶ $\lambda \geq 0$ is a *tuning parameter* regulating the relative importance of the two terms.
- ▶ $\lambda = 0 \implies$ OLS
- ▶ $\lambda \rightarrow \infty \implies$ Model with no regressors

Equivalent formulation

$$\min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\}$$

subject to $\sum_{j=1}^p \beta_j^2 \leq s$

- ▶ What happens when we have a change in scale on one of the X -variables?
- ▶ As an example, the restriction $\beta_1^2 + \beta_2^2 \leq 1$ is not the same for the situations where x_1 is measured in 1NOK as when it is measured in 1000NOK.

Standardization of X -variables

- ▶ OLS estimates are *scale equivariant*. If we multiply, say, x_1 with 2, the OLS-estimator of β_1 will be reduced by half.
 - ▶ $Y = \beta_0 + \beta_1 X_1 + \varepsilon$
 - ▶ $Y = \beta_0 + \gamma_1 (2X_1) + \varepsilon$
 - ▶ A regression of Y on $X'_1 = 2X_1$ will yield $\hat{\gamma}_1^{OLS} = \hat{\beta}_1^{OLS} / 2$
 - ▶ This is not the case in Ridge regression
- ▶ Therefore we make sure that all variables are of the same scale (variance one).
- ▶ Instead of regressing on x_{ij} , regress on

$$\tilde{x}_{ij} = \frac{x_{ij}}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x})^2}}$$

Does it work

Example: Simulate $n = 100$ observations from the model

$$y = 1 + 2x_1 + 3x_2 + 4x_3 + \dots + 11x_{10} + \varepsilon,$$

where x_j are generated from $N(0, j^2)$ -distributions and ε from a $N(0, 100^2)$ -distribution.

```
set.seed(1234567)
n=100
e=rnorm(n,sd=200)
beta=2:11
j=1:10
X=matrix(0,n,10)
for(j in 1:10) X[,j]=rnorm(n,sd=j)
y=1+X%*%beta+e
```

```
head(X)
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,] -0.1858555  2.670053 -0.96498936 -3.7996303  -6.213529 12.746656
## [2,]  0.6234541 -1.007334  1.07425972 -2.9030187 -11.731485  9.586391
## [3,] -0.8221098 -2.345024  0.07530799 -1.9510323  2.069667 -4.741907
## [4,]  1.6184734  2.490086  4.67716457  1.0656960 -2.389036 -2.821774
## [5,] -0.2177257  3.109601  1.86177590 -0.3940348 -2.044731  7.258113
## [6,] -0.1005853  2.103262  0.72068924  3.5212289 -3.700552 -4.254479
##           [,7]      [,8]      [,9]     [,10]
## [1,]  0.9738641  8.7207091 -11.294981 -3.8736312
## [2,] -7.0785018 -2.2240687  1.962232 -4.4666371
## [3,]  5.3929717  0.7773555 -3.312801 -0.0806408
## [4,] -5.3418241  8.1699491 19.521598  1.6780237
## [5,] -4.4330567 -6.8395802  8.817132  5.4906659
## [6,]  0.5514795 -0.1474277 13.426321  0.9224953
```

```
head(y)
```

```
##           [,1]
## [1,] -0.2159528
## [2,] 154.3296007
## [3,] 124.3503982
## [4,] -24.0332804
## [5,] 103.7536544
## [6,] 187.3048668
```

OLS

```
##
## Call:
## lm(formula = y ~ X)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -427.38 -118.06   15.73  128.38  440.19
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -16.854     19.872  -0.848  0.39863
## X1           -18.004     20.247  -0.889  0.37629
## X2             10.137      9.172   1.105  0.27204
## X3             3.508      7.659   0.458  0.64801
## X4             4.793      5.672   0.845  0.40042
## X5             9.669      3.850   2.511  0.01383 *
## X6             9.585      3.238   2.960  0.00395 **
## X7             3.848      3.461   1.112  0.26925
## X8             6.521      2.915   2.237  0.02779 *
## X9            11.100      2.113   5.253 1.01e-06 ***
## X10           12.707      2.222   5.718 1.42e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 196.1 on 89 degrees of freedom
## Multiple R-squared:  0.515, Adjusted R-squared:  0.4606
## F-statistic: 9.452 on 10 and 89 DF,  p-value: 1.604e-10
```


Does Ridge regression improve predictions?

Let $\lambda = 0.01$ (we will come back to this)

Remember: We minimize $RSS(\beta) + \lambda \sum_{k=1}^p \beta_k^2$

```
require(glmnet)
ridge=glmnet(X,y,alpha=0,lambda=0.01)
cbind(coef(ols),coef(ridge))
```

```
## 11 x 2 sparse Matrix of class "dgCMatrix"
##                               s0
## (Intercept) -16.854251 -16.853338
## X1          -18.003679 -18.000938
## X2           10.137177  10.137581
## X3           3.508391   3.507846
## X4           4.792734   4.791863
## X5           9.669174   9.669263
## X6           9.584630   9.583842
## X7           3.848192   3.848173
## X8           6.520966   6.520296
## X9          11.100250  11.099486
## X10         12.707348  12.706911
```

Does Ridge regression improve predictions?

$\lambda = 100000$

```
require(glmnet)
ridge=glmnet(X,y,alpha=0,lambda=1000000)
cbind(coef(ols),coef(ridge))
```

```
## 11 x 2 sparse Matrix of class "dgCMatrix"
##                               s0
## (Intercept) -16.854251 -7.1471782714
## X1          -18.003679  0.0004085510
## X2           10.137177  0.0069577694
## X3            3.508391  0.0005011873
## X4            4.792734  0.0006766484
## X5            9.669174  0.0039692326
## X6            9.584630  0.0016825711
## X7            3.848192  0.0027021290
## X8            6.520966  0.0013241532
## X9           11.100250  0.0020376406
## X10          12.707348  0.0037612079
```

```
mean(y)
```

```
## [1] -7.14598
```

Choosing λ by LOO cross-validation

```
lambdamin=cv.glmnet(X,y,alpha=0,nfold=n)$lambda.min  
lambdamin
```

```
## [1] 28.27952
```

```
ridgemin=glmnet(X,y,alpha=0,lambda=lambdamin)  
cbind(coef(ols),coef(ridgemin))
```

```
## 11 x 2 sparse Matrix of class "dgCMatrix"
```

```
##                                s0
```

```
## (Intercept) -16.854251 -15.208627
```

```
## X1          -18.003679 -14.006789
```

```
## X2           10.137177  10.601216
```

```
## X3           3.508391   2.878702
```

```
## X4           4.792734   3.821444
```

```
## X5           9.669174   9.225791
```

```
## X6           9.584630   8.167970
```

```
## X7           3.848192   3.974585
```

```
## X8           6.520966   5.524961
```

```
## X9          11.100250   9.508798
```

```
## X10         12.707348  11.498724
```

Test predictions

```
xy=data.frame(X,y)
train=xy[1:floor(n/2),]
test=xy[(floor(n/2)+1):n,]
Xtrain=X[1:floor(n/2),]
ytrain=y[1:floor(n/2)]
Xtest=X[(floor(n/2)+1):n,]
ytest=y[(floor(n/2)+1):n]
olstrain=lm(y~.,data=train)
predols=predict(olstrain,newdata=test)
olsERR=ytest-predols
ridgetrain=glmnet(Xtrain,ytrain,alpha=0,lambda=lambdamin)
predridge=predict(ridgetrain,newx=Xtest)
ridgeERR=ytest-predridge
mean(olsERR^2)
```

```
## [1] 53559.83
```

```
mean(ridgeERR^2)
```

```
## [1] 52444.16
```

Lasso

- Minimize

$$\underbrace{\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2}_{\text{Fit to data}} + \lambda \underbrace{\sum_{k=1}^p |\beta_k|}_{\text{Shrinkage penalty}}$$

- In contrast to Ridge regression, the solution to the Lasso-problem can result in some coefficient estimates exactly equal to zero.
- Variable selection approach?

Choosing λ in Lasso by LOO cross-validation

```
lassolambda=cv.glmnet(X,y,alpha=1,nfold=n)$lambda.min  
lassolambda
```

```
## [1] 13.12619
```

```
lasso=glmnet(X,y,alpha=1,lambda=lassolambda)  
cbind(coef(ols),coef(ridgemin),coef(lasso))
```

```
## 11 x 3 sparse Matrix of class "dgCMatrix"  
##                               s0          s0  
## (Intercept) -16.854251 -15.208627 -14.323654  
## X1          -18.003679 -14.006789  .  
## X2           10.137177  10.601216  5.087692  
## X3           3.508391   2.878702  .  
## X4           4.792734   3.821444  .  
## X5           9.669174   9.225791  8.254820  
## X6           9.584630   8.167970  6.560264  
## X7           3.848192   3.974585  2.374277  
## X8           6.520966   5.524961  3.372967  
## X9          11.100250   9.508798  8.562229  
## X10         12.707348  11.498724 12.268895
```

Test predictions

```
Xtrain=X[1:floor(n/2),]  
ytrain=y[1:floor(n/2)]  
Xtest=X[(floor(n/2)+1):n,]  
ytest=y[(floor(n/2)+1):n]  
lassotrain=glmnet(Xtrain,ytrain,alpha=1,lambda=lassolambda)  
predlasso=predict(lassotrain,newx=Xtest)  
lassoERR=ytest-predlasso  
mean(olsERR^2)
```

```
## [1] 53559.83
```

```
mean(ridgeERR^2)
```

```
## [1] 52444.16
```

```
mean(lassoERR^2)
```

```
## [1] 51258.12
```

Dimension reduction by Principal Component Analysis (PCA)

- ▶ $X_1, X_2, \dots, X_p$ are our original predictors.
- ▶ $Z_1, Z_2, \dots, Z_M$, where $M < p$ are linear combinations of the original predictors.
- ▶ $Z_m = \sum_{j=1}^p \phi_{jm} X_j$, $m = 1, \dots, M$
- ▶ If the ϕ 's are chosen well, y might be better predicted by the Z 's than by the X 's.
- ▶ We reduce the dimension of the predictor space from p to M .

Principal Component Analysis (PCA)

- ▶ Example: X_1 and X_2 are the scores on two school tests. Let $Z = \lambda X_1 + (1 - \lambda)X_2$. In order to distinguish as well as possible between students based on the two scores, find the λ that maximizes the variance of Z , $\text{Var}(Z) = g(\lambda)$.



$$\begin{aligned}g(\lambda) &= \lambda^2 \sigma_1^2 + (1 - \lambda)^2 \sigma_2^2 + 2\lambda(1 - \lambda)\sigma_{12} \\g'(\lambda) &= 2\lambda\sigma_1^2 - 2(1 - \lambda)\sigma_2^2 + 2(1 - 2\lambda)\sigma_{12} = 0 \\ \lambda(\sigma_1^2 + \sigma_2^2 + 2\sigma_{12}) - \sigma_2^2 + \sigma_{12} &= 0\end{aligned}$$



$$\lambda_{\max} = \frac{\sigma_2^2 - \sigma_{12}}{\sigma_1^2 + \sigma_2^2 - 2\sigma_{12}}$$

- ▶ The linear combination that maximizes the variance is $Z_{\max} = \lambda_{\max}X_1 + (1 - \lambda_{\max})X_2$

Principal Component Analysis (PCA)

- ▶ $Z_{\max}$ is the *first principal component*.
- ▶ The *second principal component* is the linear combination, uncorrelated with $Z_{\max}$, which has the highest variance.
- ▶ etc
- ▶ The Principal Components Regression (PCR) Approach is the idea of finding the first M principal components and use them as a substitute for the p predictors.
- ▶ If most of the variation in the predictors can be seen in just a few principal components we estimate fewer parameters and thereby reduce the overfitting problem.

The Auto data

```
library(pls)
library(ISLR)
age=83-Auto$year
Auto2=Auto[,!(names(Auto) %in% c("name","origin","year"))]
Auto2$age=age
pcr.fit=pcr(mpg~.,data=Auto2)
summary(pcr.fit)
```

```
## Data:      X dimension: 392 6
## Y dimension: 392 1
## Fit method: svdpc
## Number of components considered: 6
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## X      99.76   99.96   100.00   100.00   100.00   100.00
## mpg     69.35   70.09   70.75   80.79   80.88   80.93
```

The first principal component explains 99.8% of the variation in all 6 X-variables.

Use these two as regressors

```
regall=lm(mpg~.,data=Auto2)
summary(regall)
```

```
##
## Call:
## lm(formula = mpg ~ ., data = Auto2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.6927 -2.3864 -0.0801  2.0291 14.3607
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  47.9942254   2.1625355   22.193  <2e-16 ***
## cylinders    -0.3298591   0.3321041   -0.993   0.321
## displacement  0.0076784   0.0073577    1.044   0.297
## horsepower   -0.0003914   0.0138365   -0.028   0.977
## weight       -0.0067946   0.0006700  -10.141  <2e-16 ***
## acceleration  0.0852732   0.1020356    0.836   0.404
## age          -0.7533672   0.0526181  -14.318  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.435 on 385 degrees of freedom
## Multiple R-squared:  0.8093, Adjusted R-squared:  0.8063
## F-statistic: 272.2 on 6 and 385 DF, p-value: < 2.2e-16
```

Training/test evaluation

```
n=nrow(Auto2)
ntrain=floor(n/2)
ind=sample(1:n,size=ntrain)
train=Auto2[ind,]
test=Auto2[-ind,]
regall=lm(mpg~.,data=train)
pcr.fit=pcr(mpg~.,ncomp=4,data=train)
predall=predict(regall,newdata=test)
predpcr=predict(pcr.fit,newdata=test)
MSEall=mean((test$mpg-predall)^2)
MSEpcr=mean((test$mpg-predpcr)^2)
MSEall
```

```
## [1] 11.47451
```

```
MSEpcr
```

```
## [1] 15.59672
```